Relazione - Versione 11: Tokenizzazione Subword e Dataset Pubblico

Introduzione

La Versione 11 mantiene il piccolo Transformer decoder-only introdotto nella Versione 10 e cambia il modo in cui il linguaggio entra nel modello. I due Transformer block causali riutilizzabili, le connessioni residuali, le operazioni LayerNorm, le reti feed-forward e l'obiettivo di predizione del token successivo vengono preservati.

Il minuscolo corpus hardcoded a caratteri viene sostituito dal dataset pubblico WikiText-103 Raw v1 e gli identificatori di carattere vengono sostituiti da identificatori di token subword ottenuti con Byte Pair Encoding (BPE). I confini originali degli split training, validation e test vengono preservati, mentre subset controllati mantengono l'esperimento abbastanza piccolo da restare ispezionabile e riproducibile.

```
Testo WikiText-103
  |
Tokenizer BPE addestrato sul subset di training selezionato
  |
Vocabolario di 2048 token
  |
Contesto di 16 token + 16 target sequenziali
  |
Embedding token + posizione
  |
Transformer block 1
  |
Transformer block 2
  |
Logits del vocabolario a ogni posizione
  |
Predizione del token successivo e generazione greedy
```

Obiettivo della Versione 11

- sostituire il minuscolo corpus hardcoded a caratteri con un dataset pubblico documentato per il language modeling;
- sostituire la tokenizzazione character-level con un tokenizer BPE subword addestrato solo sul subset di training selezionato;
- preservare i confini ufficiali degli split training, validation e test di WikiText;
- costruire sequenze di token a lunghezza fissa per la predizione del token successivo senza attraversare i confini tra righe di testo;
- mantenere la stessa architettura Transformer decoder-only a due blocchi sviluppata nella Versione 10;
- valutare separatamente le loss di training, validation e test e ispezionare predizioni BPE, generazione e causal attention;
- mantenere l'implementazione esplicita e computazionalmente piccola introducendo il language modeling subword su dati pubblici.

Architettura

Il flusso completo della Versione 11 è:

```
Dataset: WikiText-103 Raw v1
  |
Righe originali: train 1,801,350 / validation 3,760 / test 4,358
  |
Righe selezionate: train 10,000 / validation 1,000 / test 1,000
  |
Vocabolario BPE: 2048 token
  |
Finestre grezze a lunghezza fissa: 96,893 / 9,764 / 9,572
  |
Sequenze usate: train 12,000 / validation 2,000 / test 2,000
  |
Tensore input:  (N, 16)
Tensore target: (N, 16)
  |
Embedding token: (2048, 8) + posizioni: (16, 8)
  |
Transformer block 1: (16, 8) -> (16, 8)
  |
Transformer block 2: (16, 8) -> (16, 8)
  |
Logits del vocabolario: (16, 2048) per esempio
  |
Gradient descent mini-batch + valutazione validation/test separata
  |
Generazione autoregressiva BPE greedy
```

Dataset pubblico e subset controllati

Il notebook legge i file Parquet originali di WikiText-103 Raw v1: due shard di training, un file di validation e un file di test. Le righe di testo vuote vengono rimosse prima di campionare, con seed controllato, un subset indipendente da ciascuno split ufficiale. Non viene creato un nuovo split casuale training/validation.

La lingua del dataset è l'inglese. Informazioni su fonte, provenienza e licenza sono documentate nei metadati del dataset Kaggle collegato al notebook.

Tokenizer BPE e sequenze di token

Un tokenizer Byte Pair Encoding viene addestrato solo sul subset di training selezionato. Il tokenizer usa una dimensione massima del vocabolario di 2048, un token speciale [UNK] e il suffisso </w> per segnare la fine di una parola pre-tokenizzata. I testi di validation e test non partecipano alla costruzione del vocabolario.

Ogni riga WikiText viene codificata indipendentemente. Le finestre contengono CONTEXT_LENGTH + 1 = 17 token: i primi 16 token formano l'input e la stessa sequenza spostata di una posizione forma il target di 16 token. Uno stride di 16 evita sovrapposizioni eccessive, le finestre finali incomplete vengono scartate e gli esempi non attraversano mai i confini tra righe.

Parametri addestrabili

Il modello contiene 23 variabili TensorFlow addestrabili e precisamente 34,496 parametri addestrabili:

```
Embedding token                 2048 x 8    = 16384
Embedding posizionali            16 x 8    =   128
Transformer block 1                         =   800
Transformer block 2                         =   800
Pesi output                    8 x 2048   = 16384
-------------------------------------------------
Totale                                     = 34496
```

Ogni Transformer block continua a contribuire esattamente 800 parametri:

```
Proiezione query                 8 x 16 = 128
Proiezione key                   8 x 16 = 128
Proiezione value                 8 x 16 = 128
Proiezione output attention     16 x 8  = 128
LayerNorm 1 scala + shift        8 + 8   =  16
Input feed-forward               8 x 16 = 128
Output feed-forward             16 x 8  = 128
LayerNorm 2 scala + shift        8 + 8   =  16
-----------------------------------------------
Per blocco                                 = 800
```

Dettagli del Transformer block

La matematica interna del Transformer viene preservata dalla Versione 10. Per una rappresentazione di input X, ogni blocco calcola proiezioni query, key e value indipendenti, applica causal scaled dot-product attention, proietta il risultato dell'attention nuovamente nella rappresentazione del modello a 8 dimensioni e poi applica connessioni residuali, LayerNorm e una rete feed-forward position-wise 8 -> 16 -> 8.

```
Q = X Wq
K = X Wk
V = X Wv

scores = Q K^T / sqrt(dk)
masked_scores = causal_mask(scores)
A = softmax(masked_scores)
H = A V
```

Non vengono usati termini di bias nelle proiezioni di attention, feed-forward o vocabolario. Entrambe le operazioni LayerNorm mantengono parametri addestrabili di scala e shift.

Causal mask

```
[[1, 0, 0, ..., 0],
 [1, 1, 0, ..., 0],
 [1, 1, 1, ..., 0],
 ...
 [1, 1, 1, ..., 1]]
```

Loss, training e riproducibilità

La sparse softmax cross-entropy viene calcolata in tutte le 16 posizioni della sequenza. La loss di ogni mini-batch è la media sui relativi target token e le loss a livello di dataset vengono accumulate pesando per il numero di token, così da gestire correttamente l'ultimo batch parziale.

tf.GradientTape calcola i gradienti per tutte le 23 variabili addestrabili. I lookup degli embedding dei token possono produrre gradienti sparsi IndexedSlices; questi vengono aggiornati direttamente con scatter_sub, mentre i gradienti densi usano assign_sub.

```
Learning rate: 1.0
Batch size:    32
Epoche:        20
Seed:          42

Sequenze training:     12,000  -> 375 batch
Sequenze validation:    2,000  ->  63 batch
Sequenze test:          2,000  ->  63 batch
```

Risultato del training salvato

```
Loss training iniziale:       7.626767
Loss training finale:         4.987922
Loss validation iniziale:     7.626187
Loss validation finale:       5.085637
Loss test:                     5.117153
Baseline vocabolario uniforme: 7.624619
```

Nell'esecuzione salvata, le loss di training e validation scendono nettamente sotto la baseline del vocabolario uniforme e il subset di test indipendente selezionato produce una loss vicina alla validation. Questi valori descrivono questa esecuzione salvata; tra ambienti TensorFlow possono comparire piccole differenze numeriche.

Ispezione delle probabilità apprese del token successivo

Per il contesto di test non visto "Little Gidding is the fourth and final poem", le probabilità più alte dell'ultima posizione nell'esecuzione salvata sono:

```
','      0.1392
'.'      0.0825
'and'    0.0471
'of'     0.0453
"'"      0.0336
'('      0.0253
'was'    0.0252
```

Il vero token successivo è "of</w>" con probabilità del modello 0.0453, che lo colloca tra i candidati con probabilità più alta in questa esecuzione.

Ispezione della causal attention impilata

L'attention dell'ultima posizione del Block 1 è quasi uniforme sui 16 token di contesto, con pesi vicini a 0.0625. Il Block 2 è più selettivo; i suoi maggiori pesi salvati includono G = 0.2545, L = 0.1621, f = 0.0861 e the = 0.0600. La somma dell'attention dell'ultima posizione è 1.000000 in entrambi i blocchi, mentre i test completi confermano attention nulla verso le posizioni future.

Generazione autoregressiva

La generazione parte da un prompt di testo, lo codifica con lo stesso tokenizer BPE, mantiene come contesto i 16 identificatori di token più recenti e seleziona ripetutamente il token successivo con probabilità più alta. La Versione 11 usa quindi greedy decoding deterministico invece di campionamento stocastico.

La dimostrazione salvata parte da:

```
Little Gidding is the fourth and final poem
```

e genera 40 token BPE aggiuntivi. L'output decodificato salvato è:

```
Little Gidding is the fourth and final poem , and the Sea , and the Sea , and the Sea
, and the Sea , and the Sea , and the Sea , and the S
```

La frase ripetuta è coerente con il modello volutamente piccolo, il contesto di 16 token e la strategia di selezione greedy. Poiché la pre-tokenizzazione whitespace tratta la punteggiatura come unità separate, il testo decodificato può anche contenere spazi prima della punteggiatura; si tratta di un effetto di formattazione del tokenizer e non di un errore del Transformer.

Controlli inclusi

- la dimensione del vocabolario del tokenizer è esattamente 2048;
- i tensori input/target di training, validation e test hanno la lunghezza configurata di 16 token e le dimensioni massime previste per ciascuno split;
- gli identificatori dei token restano all'interno del vocabolario e la prima sequenza target corrisponde all'input spostato di un token;
- l'output del modello ha forma (1, 16, 8), i logits hanno forma (1, 16, 2048) ed entrambe le matrici di attention hanno forma (16, 16);
- il modello contiene due oggetti Transformer block indipendenti, 23 variabili addestrabili ed esattamente 34,496 parametri addestrabili;
- ogni riga di attention somma a 1 e tutti i pesi di attention verso posizioni future sono zero;
- le cronologie delle loss di training e validation contengono 20 valori finiti e le loss finali di training e validation migliorano rispetto ai valori iniziali e restano sotto la baseline del vocabolario uniforme;
- la loss di test è finita e sotto la baseline del vocabolario uniforme;
- la distribuzione di probabilità del token successivo ispezionata è finita e somma a 1;
- la generazione autoregressiva aggiunge esattamente 40 nuovi token BPE.

La sezione di test del notebook termina con All checks passed.

Cosa insegna la Versione 11

La Versione 11 è il primo passaggio del progetto in cui il language model passa da una minuscola dimostrazione character-level a una pipeline subword su testo pubblico, preservando il Transformer decoder-only sviluppato nella Versione 10. La progressione concettuale principale è:

```
testo pubblico WikiText
        -> tokenizer BPE addestrato sui dati di training
        -> identificatori di token subword
        -> contesti fissi di 16 token e target traslati
        -> embedding token + posizione
        -> Transformer block 1
        -> Transformer block 2
        -> logits su vocabolario di 2048 token
        -> probabilità del token successivo
        -> generazione BPE greedy
```

La differenza importante rispetto alla Versione 10 non è una nuova equazione di attention. Il nucleo Transformer resta volutamente invariato, così da isolare e ispezionare direttamente l'effetto della nuova fonte dati e della nuova rappresentazione del linguaggio.

Limitazioni

- vengono usati solo subset controllati di WikiText-103, quindi l'esperimento non tenta un training sull'intero dataset;
- il modello resta intenzionalmente minuscolo, con rappresentazioni a 8 dimensioni, due Transformer block e un solo meccanismo di attention per blocco invece della multi-head attention;
- la lunghezza del contesto è fissa a 16 token BPE;
- la configurazione BPE basata su whitespace è volutamente semplice e può introdurre spazi prima della punteggiatura durante la decodifica;
- il greedy decoding è deterministico e può amplificare pattern di generazione ripetitivi;
- il training mantiene i parametri dell'epoca finale e non implementa ancora salvataggio dei checkpoint, ripristino o logica di resume;
- non sono presenti learning-rate schedule, dropout, mixed precision o altre infrastrutture di training su scala maggiore;
- la valutazione resta volutamente leggera; strategie di generazione controllata e valutazioni più sistematiche vengono lasciate alle versioni successive.

Conclusione

La Versione 11 sostituisce con successo la pipeline di input character-level con un tokenizer BPE addestrato su un dataset pubblico documentato, preservando il Transformer decoder-only causale a due blocchi della Versione 10. I confini degli split ufficiali restano separati, le sequenze subword vengono create indipendentemente all'interno delle righe WikiText e il modello ora predice su un vocabolario di 2048 token invece di 27 caratteri.

Il modello resta volutamente piccolo, ma il progetto dispone ora della rappresentazione dei dati e della pipeline su corpus pubblico necessarie per la fase successiva. La Versione 12 può costruire su questa base introducendo una pipeline di training dedicata e checkpoint senza modificare la transizione concettuale fondamentale completata qui.